

Complete Coverage Algorithm Based on Linked Smooth Spiral Paths for Mobile Robots

Tae-Kyeong Lee Sang-Hoon Baek Se-Young Oh

Electronic and Electrical Engineering Department
Pohang University of Science and Technology
(POSTECH) Pohang, Korea
{devilee, tkeb100, syoh}@postech.ac.kr

Young-Ho Choi

Fusion Robot Team, Dept. of R&D
Pohang Institute of Intelligent Robotics (PIRO)
Pohang, Korea
rockboy@postech.ac.kr

Abstract—This paper presents an on-line complete-coverage path planning algorithm for mobile robots based on approximate cellular decomposition, which abstracts the target environment using grid. Most existing grid-based coverage algorithms have a common problem of constrained mobility which degrades the efficiency of the coverage task by inducing zigzag like path. In this paper, we propose a new complete coverage path generation algorithm, linked-smooth-spiral-path (LSSP), which removes the constraint on mobility by adopting a high-resolution grid-map representation of the environment and a cardinal-spline curve-model to generate a smooth spiral coverage path. We define a new performance measure for coverage tasks, which quantifies the smoothness of the coverage path. Simulation results demonstrate the improved coverage performance of the proposed algorithm compared to other existing grid-based coverage algorithms.

Keywords—coverage path planning, smooth spiral path, mobile robots.

I. INTRODUCTION

Coverage path planning algorithms have received much attention in the past several decades because of its wide applications in robotics, including floor cleaning [1], lawn mowing [2], de-mining [3], and harvesting [4]. These algorithms determine a path that enables a sweeping tool attached to the robot to pass through all free points in a closed target environment, while maximizing coverage and minimizing energy and time consumption.

Some of the early approaches in coverage path planning rely on heuristics and randomized components [5]. These approaches are applicable in real robots, but they do not guarantee complete coverage of the target environment. Assuming that a map for the environment is given a priori, some algorithms find the optimal complete-coverage path off-line [6]. However, exact information about the environment is usually not available and the environment is not fixed, hence many on-line sensor based coverage path planners have been developed [7]. Many complete-coverage path planners break down the target region into cells such that coverage in each cell is simple. This kind of method is called a cellular

decomposition [8]. Approximate cellular decomposition represents the target environment with a union of fine-grid cells, which are all the same size and shape [9]. Semi-approximate cellular decomposition partially discretizes the space with cells which have fixed width but can have any shape at the top and bottom [10]. An exact cellular decomposition is the set of non-intersecting cells, whose union fills the target environment [11].

As a filling motion, back-and-forth motion has been widely used [12], [13]. Some of other approaches, including STC [7], BSA [14], and LSP [15], are based on spiral filling motion because it is more robust to the robot's initial orientation. Most spiral filling algorithms are based on the approximate cellular decomposition, where each grid-cell is of the size of the coverage tool. To ensure that each cell is covered in single step movement, they quantize the moving direction of the robot. Due to this constrained mobility, the number of the turns in the trajectory is significantly affected by the initial robot pose and the shape of the target environment [15]. The number of turns is closely related with the efficiency of the coverage task because turning consumes much time and energy. LSP algorithm [15] tries to reduce the dependency of performance on the initial pose by aligning the map coordinates parallel to the wall, but it requires the assumptions that the robot starts near the wall and that the environment is rectangular. Oh [16] suggests to use triangular-cell-based map to increase the mobility. However, this concept is used only for smoothing start-goal-navigation path, which does not significantly improve the performance of the overall coverage task.

This paper presents a new coverage path planning algorithm which eliminates constraint on mobility and generates smooth coverage paths. This method is based on a high-resolution grid-map representation and utilizes a cardinal-spline curve-model to generate a continuous smooth-spiral-path. The paths are linked through constrained inverse distance transform (CIDT) [15] to guarantee completeness of the coverage. To compare the performance of the proposed algorithm with existing coverage algorithms, we suggest a new measure which quantifies the smoothness of a coverage path by approximating the average curvature of the path.

The rest of this paper is organized as follows. Section II addresses problems related to low-resolution grid-map representation, and suggests high-resolution grid-map

This research was supported by the MKE(The Ministry of Knowledge Economy), Korea, under the Core Technology Development for Breakthrough of Robot Vision Research support program supervised by the NIPA(National IT Industry Promotion Agency) (NIPA-2010-C7000-1001-0006)

representation. Based on this representation, the proposed coverage algorithm is described in section III. We define a new measure of coverage-path smoothness in section IV. Finally, simulation results are shown in section V with performance comparison with other algorithms.

II. GRID-MAP REPRESENTATIONS

A. Low-Resolution Grid-Map Representation

Most existing algorithms based on approximate cellular decomposition, including STC [7], BSA [14], and LSP [15], use low-resolution grid-map representation (Fig. 1 (a)), where the size of each cell is the same as the sweeping tool. In this representation, choosing the robot's next position in the map is very simple. It only needs to choose one of the four neighborhood cells. However, this quantization results in some problems. To make such a sharp turn, the robot should stop first, which is inefficient because it slows down the coverage process and consumes energy due to deceleration and acceleration. During a coverage process, when a wall or an obstacle is not aligned parallel to one of the quantized directions, a zigzag path is generated as illustrated in Fig. 2. The zigzag path increases the number of turns in the whole path. Therefore, the number of turns is correlated with the map coordinates. Nevertheless, most existing algorithms do not consider this problem much and just assume that the robot is already aligned to the wall and set the map coordinates using the initial robot pose. Indeed, it is usually impossible to align the map coordinate parallel to every obstacle in the environment.

Coarse resolution of the grid-map and the constrained mobility also prevent the coverage of partially-occupied cells as shown in Fig. 2. Many of the approximate cellular decomposition methods including STC do not provide solutions to this problem. To handle this, wall-following behavior has been suggested in extended BSA algorithm [14]. This behavior is a purely sensor-based reactive navigation method, which ignores the constraint put forward by the grid decomposition. By using this method, the partially-occupied cells around obstacles could be effectively covered. However, some cells are only partially covered during wall-following behavior. LSP algorithm covers those cells again using spiral filling rules. This approach is reasonable but increases the overlapping regions.

B. High-Resolution Grid-Map Representation

To increase the mobility, we suggest using a high-resolution grid-map (Fig. 1 (b)), where the tool is bigger than each grid-cell by a few times. In this representation, choosing the robot's next position is much more complex because there are more neighborhood cells than in the low-resolution case. However, the mobility of the robot is increased naturally because the quantization level is lowered by extended choices for the next target pose.

To record the sensory information on the map, we define four types of cells: unexplored cell, covered cell, candidate cell, and obstacle cell. The unexplored cell is the initial setting for the map, where the robot has not visited or investigated yet. The covered cell is the visited cell. Here, we assume that the coverage-tool is a line-segment and is attached under the center

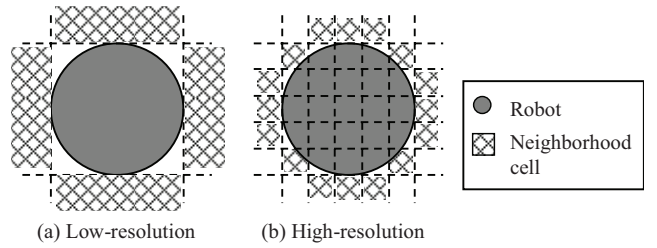


Figure 1. Grid-map representations of the environment.

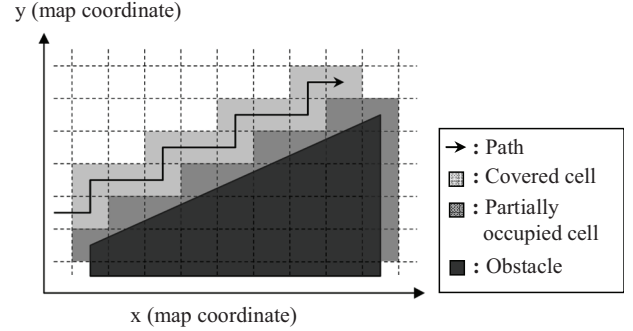


Figure 2. Zigzag path due to the low-resolution grid-map representation and the constrained mobility.

of the robot perpendicular to the robot's heading direction. During the coverage process, only the cells under this line-segment are marked as covered cells. A candidate cell is an obstacle-free cell investigated by the range sensors attached to the robot. Finally, an obstacle cell is one occupied by obstacles.

III. LINKED-SMOOTH-SPIRAL-PATH

The proposed linked-smooth-spiral-path (LSSP) algorithm is based on a high-resolution grid-map representation, which is presented in section II. It consists of two main components: the smooth-spiral-path (SSP) generation rule, and the constrained inverse distance transform (CIDT). The former method generates a simple SSP that covers some part of the whole environment. The latter algorithm is used to find an uncovered region and generate a linkage path between old and new SSPs.

A. Smooth-Spiral-Path (SSP) Generation Rule

To cover an unvisited free region, we suggest an efficient path generation rule, which generates a smooth spiral path. The overall process of the rule is shown in Fig. 3. To generate spiral-like path, the covered cells are treated as a virtual wall, and the robot successively follows the left side of both the virtual wall and real obstacles until every adjacent cell is covered. When there is no virtual wall or real obstacle in the desired position, like just after the coverage started, the robot just turns clockwise following a circle whose radius is half the length of the coverage-tool. After the virtual wall is generated, the robot can follow it.

The SSP generation rule has three behavior components, which are shown in gray boxes in Fig. 3. Firstly, we describe the virtual-wall-following behavior, which is composed of three steps. The first step is to construct a parametric curve model of the virtual wall. Generally, we can describe the parametric curve as following vector equation:

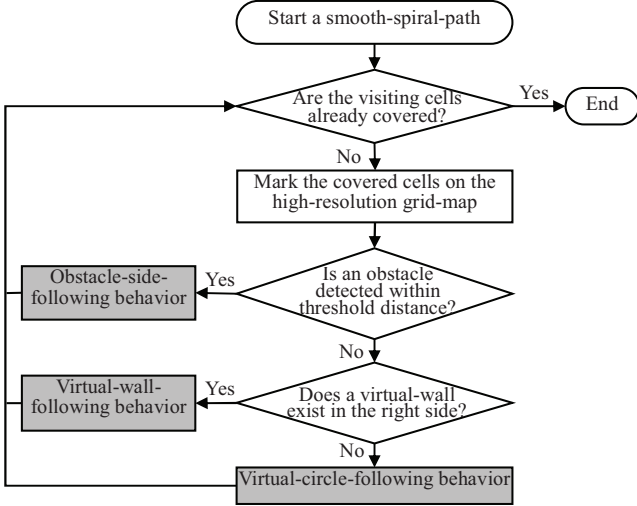


Figure 3. Flowchart of the smooth-spiral-path generation rule.

$$P(u) = \bar{a}u^3 + \bar{b}u^2 + \bar{c}u + \bar{d}, \quad (0 \leq u \leq 1) \quad (1)$$

where P represents the point on the curve and parameter u controls where the point is, and $\bar{a}$, $\bar{b}$, $\bar{c}$, $\bar{d}$ are constant vectors that determine the shape of the curve. There are several approaches to get the shape parameters [17]. Here, we adopt a cardinal-spline curve-model [17] because of its useful characteristics. Given sequence of control points, the cardinal splines interpolate piecewise cubic polynomials, where C^2 continuous curve sections are connected as C^1 continuous as shown in Fig. 4. The advantage is that given a list of any number of control points, we can generate single smooth curve passing through all the control points. Also, we do not have to give the values for the endpoint tangents because they are calculated from the two adjacent control points. This makes it simple to construct a continuous curve model sequentially, in the situation of control points being sampled on-line.

A cardinal-spline section is completely specified with four consecutive control points. The middle two control points are the section endpoints, and the other two points are used in the calculation of the endpoint slopes. If we construct a curve section between control points p_k and p_{k+1} , the four control points from p_{k-1} to p_{k+2} are used to set the boundary conditions for the cardinal spline section as:

$$P_k(0) = p_k, \quad (2)$$

$$P_k(1) = p_{k+1}, \quad (3)$$

$$P_k'(0) = \frac{1}{2}(1-t)(p_{k+1} - p_{k-1}), \quad (4)$$

$$P_k'(1) = \frac{1}{2}(1-t)(p_{k+2} - p_k), \quad (5)$$

where t is the tension parameter between 0 and 1, which controls how loosely or tightly the cardinal-spline fits the input control points as depicted in Fig. 4. We set the tension parameter to 0.5, which is the middle of the range of t . Using these boundary conditions, we can easily calculate the shape parameters in (1). However, we have only three control-points

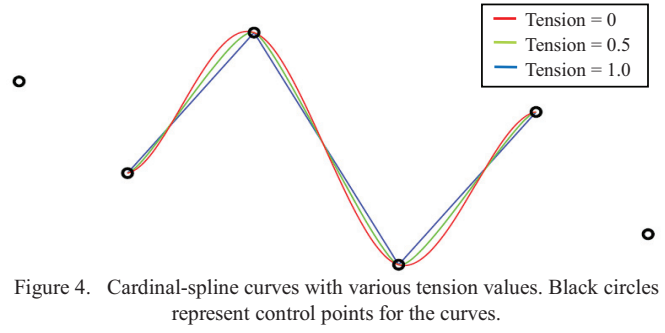


Figure 4. Cardinal-spline curves with various tension values. Black circles represent control points for the curves.

for constructing the first curve section, i.e. $k=0$. In this case, we use a different boundary condition for $P_0'(0)$:

$$P_0'(0) = \frac{1}{2}(1-t)(p_1 - p_0). \quad (6)$$

The control-points for the curve-model are sampled from covered cells at the border between covered and uncovered regions depicted in Fig. 5, and stored in a control-point stack (CPS). The CPS is implemented as a first-in-first-out (FIFO) stack which stores six control points, by which at most three curve sections are completely described as shown in Fig. 4. When CPS is empty, the nearest covered cell at the border is selected as the first sample point. Then next points are sampled along the border until the CPS is full. The sampling distance determines fitting level of the curve-model. We empirically set this value as two thirds of the tool-length.

The generated curve model is used to calculate a target pose for the robot to follow, which is the second step of the virtual-wall-following behavior. A method for generating the target pose is depicted in Fig. 5. In this figure, C_1 and r represent the current robot pose and the half length of the coverage tool, respectively. First, we find a point P_1 on the curve which is the nearest point from C_1 . Here, we define an empirical parameter L , which is a step length between the current position and the target. It requires much computational resource to calculate a point on curve that is L distance away from a certain point. Therefore, to reduce computational burden, we use an approximate method. First, an intermediate point P_2 is calculated, which is L distance away from P_1 in the tangential direction. Then P_2 is projected on the parametric curve to get the point P_3 . Finally, the target pose C_2 is calculated by adding offset r to P_3 in the normal direction of the curve. The

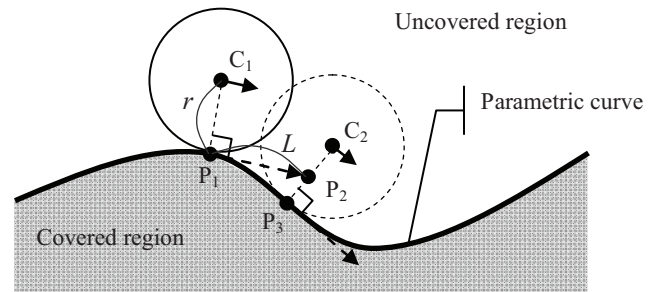


Figure 5. Target-pose generation method. (C_1 : current pose; C_2 : target pose; L : step length; P_1 , P_3 : contact points on the curve)

orientation of C_2 is set the same as the tangential direction of the curve at point P_3 . However, this approach does not work well at sharp concave corners, where C_2 is possibly close to C_1 . This situation is detected when the distance between C_1 and C_2 is smaller than a threshold distance, $0.5 \times L$. In this case, we increase the position parameter u until the distance between C_1 and C_2 is bigger than the threshold distance.

The parametric curve section $P_k(u)$ is defined in the range of $(0 \leq u \leq 1)$. If the parameter u , representing the point P_3 , exceeds the domain of definition, the first element of the CPS is taken out and a new boarder point is sampled into the CPS. Then P_3 is recalculated using a new curve section, derived from the new set of control points.

The final step of the virtual-wall-following behavior is to drive the robot to approach the target pose, C_2 , in both position and orientation. The controller is designed using a P controller, where the outputs are desired translational velocity (v) and angular velocity (ω). The rule is as follows:

$$\theta_D = k_{p1}(\theta_{C_1C_2} - \theta_{C_1}) + (1 - k_{p1})(\theta_{C_2} - \theta_{C_1}), \quad (0 \leq k_{p1} \leq 1), \quad (7)$$

$$\omega = k_{p2} \cdot \theta_D, \quad (8)$$

$$v = \min(v_{\max}, k_{p3} \cdot \text{Dist}_{C_1C_2} / \omega^2), \quad (9)$$

where $\theta_{C_1C_2}$ and $\text{Dist}_{C_1C_2}$ are the angle and distance of the line C_1C_2 , respectively; θ_{C_1} and θ_{C_2} are the orientation of C_1 and C_2 , respectively; $v_{\max}$ is the maximum velocity and 'min' is a function that returns smaller value of the two inputs; k_{p1} is a mixing coefficient; k_{p2} and k_{p3} are control gains. In (7), the first and the second terms calculate desired turning angles to approach the target position and to align with the target orientation, respectively. These two goals are combined to a single desired value, θ_D , through a weighted summation. In (9), we apply a simple velocity (v) scheduling scheme: the target velocity is proportional to the distance from the target position and inversely proportional to the squared angular velocity. After we get the desired velocities, v and ω , actual motors connected to the wheels are forced to track these desired velocities using a conventional PID controller.

The second behavior component of the SSP generation rule is the obstacle-side-following behavior, which is a sensor based reactive navigation. This behavior is started when the robot is blocked by an obstacle. The behavior ends when all the visiting cells are already covered. In our implementation, the distance between the obstacle and the robot is calculated from the range-sensor measurements, and then the robot is controlled to follow the obstacle side, preserving the distance at a certain desired value.

The third behavior component is the virtual-circle-following behavior. This behavior is processed when the CPS contains less than 3 elements, such as the beginning of the coverage path. The control law is simply that the robot only needs to preserve the following velocities:

$$v = v_c, \quad (10)$$

$$\omega = \frac{v}{r_c}, \quad (11)$$

where v_c is a constant velocity and r_c is the radius of the virtual-circle which is set to half the length of the coverage tool. When the CPS gathers more than three control-points during the virtual-circle-following, the virtual-wall-following behavior is started.

B. Constrained Inverse Distance Transform

In many cases, a simple SSP cannot cover all free regions. To guarantee the completeness, the robot is required to find and move to an uncovered region, and start another simple SSP. To link these simple paths, we adopt the constrained inverse distance transform (CIDT), which was suggested in our previous work [15]. CIDT applies distance transform starting from the current position and finds the nearest uncovered cell. Then the shortest path is found reversely from the target cell to the current cell.

As the resolution of the grid-map gets higher, CIDT requires more memory and computational cost to get a distance map. To reduce the burdens, a low-sampling process is applied to the high-resolution grid-map to generate a low-resolution grid-map. In our implementation, 5 by 5 local grid-cells on the high-resolution grid-map are gathered together to generate one cell on the low-resolution grid-map. The type of the new cell is set as the most prevailing type among the 5 by 5 cells of the high-resolution grid-map. CIDT is then applied to this low-sampled grid-map. If the CIDT finds no uncovered cell, the overall coverage task ends.

IV. NEW MEASURE OF COVERAGE-PATH SMOOTHNESS

In this section, we define a new measure to quantify the smoothness of a coverage path. If we cut the entire coverage path evenly with arbitrarily small length, l_{step} , each piece of the path can be modeled as an arc as depicted in Fig. 6. From this model, we calculate the curvature, κ , of each piece as follows:

$$\kappa_n = \frac{1}{r_n} = \frac{|\Delta\theta_n|}{l_{\text{step}}} = \frac{|\theta_{q_{n+1}} - \theta_{q_n}|}{l_{\text{step}}}, \quad (12)$$

where θ_{q_n} and $\theta_{q_{n+1}}$ represent the orientation of the start and end poses of the n_{th} piece. With this concept, we can calculate the average curvature, κ_{avg} , of the entire path as follows:

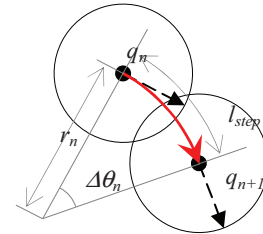


Figure 6. Modeling of the coverage path as an arc. The arrow-arc represents the n_{th} piece of the path; q_n and q_{n+1} represent the start and end robot poses.

$$N = \frac{l_{tot}}{l_{step}}, \quad (13)$$

$$\kappa_{avg} = \frac{\sum \kappa_n}{N} = \frac{\sum |\theta_{q_{n+1}} - \theta_{q_n}|}{l_{tot}}, \quad (14)$$

where l_{tot} is the total length of the entire path and N is the number of pieces. Notice that the average curvature has the same meaning with the turning angle per unit moving distance:

$$\text{Turning angle per unit moving distance} = \frac{\sum |\text{Turning angle}|}{\text{Total moving distance}}. \quad (15)$$

We use this measure to validate the smoothness of a coverage path. Smaller value of this measure represents smoother coverage path.

V. SIMULATION RESULT

Simulations were performed using our mobile robot simulator. In the simulator, we modeled a two wheeled cleaning robot as depicted in Fig. 7, which behaves similar to a physical one for given control signals. It is equipped with two IR range sensors on the side, one front IR sensor, and two front sonar sensors. The characteristics of the physical sensors were also modeled in the simulator. The diameter of the robot and the length of the tool were both set to 36 cm, and the maximal velocity of the robot was set to 35 cm/s.

A. LSSP simulation

We simulated LSSP at a cluttered environment shown in Fig. 8. The robot starts LSSP with a virtual-circle-following behavior as pictured in Fig. 8 (a). After gathering enough control points, virtual-wall-following behavior is processed. Fig. 8 (b) is the magnified view of the high-resolution grid-map around the robot during the virtual-wall-following behavior, and the path generated from this behavior becomes the spiral path in 8 (c). If the robot detects obstacles, it starts obstacle-side-following behavior as shown in Fig. 8 (c). When a simple SSP ends, a low-resolution grid-map is generated and CIDT is applied to find the nearest uncovered candidate cell as well as the path to it as depicted in Fig. 8 (d). The LSSP ends when CIDT finds no candidate cell as shown in Fig. 8 (e). At sharp turning points, some cells are not completely covered as shown in Fig. 8 (f) because of the path smoothing. They are filtered out at the low-sampling process, so the CIDT misses these regions. Although we can easily modify the low-sampling process to cover these regions, that approach increases the overlapped region. In spite of this drawback, 97 % of the free region is covered after the LSSP algorithm is applied as pictured in Fig. 8 (g). Obstacle-around areas are not complete covered because of the physical limitations.

B. Performance comparison with other algorithms

To verify the efficiency of the proposed algorithm, we simulated LSSP and three existing algorithms, STC [7], BSA [14], and LSP [15], in the same conditions. Here, the most recent or improved version of each algorithm was coded for the comparison. For the simulations, three challenging target

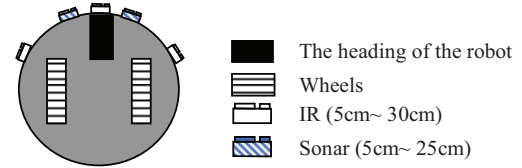


Figure 7. Configurations of our two wheeled cleaning robot equipped with 3 IR range sensors and 2 sonar range sensors.

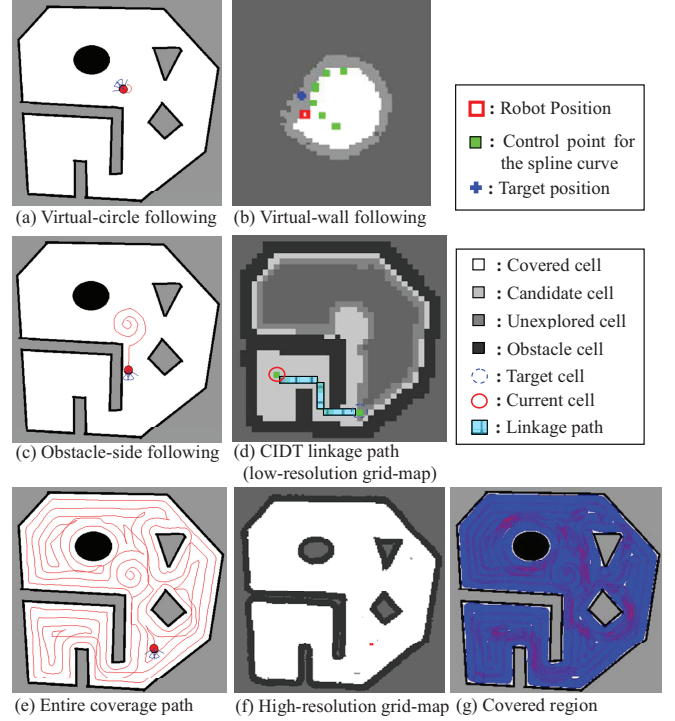


Figure 8. Simulation result of LSSP algorithm in a complex planar environment. (g) depicts covered region with blue, and overlapped region with red.

environments were designed, and for each environment, three different initial pose were selected randomly to get generalized performance of the algorithms. Fig. 9 shows coverage paths of the algorithms after conducting one of the nine set of simulations.

For each simulation, we measured three quantities: turning angle per unit moving distance, coverage rate, and overlap rate. The first quantity was calculated using (15). The other quantities were evaluated by the following equations:

$$\text{Coverage rate} = \frac{\text{Covered region}}{\text{Unoccupied region}} \times 100\%, \quad (16)$$

$$\text{Overlap rate} = \frac{\text{Overlapped region}}{\text{Covered region}} \times 100\%. \quad (17)$$

After all the simulations, the mean and standard deviation of each quantity were calculated (see Table I). Turning angle per unit moving distance was the lowest in LSSP. This implies that the LSSP generates the smoothest path among the four algorithms. LSSP also established the highest coverage rate, recording near a hundred percent. The coverage performance of

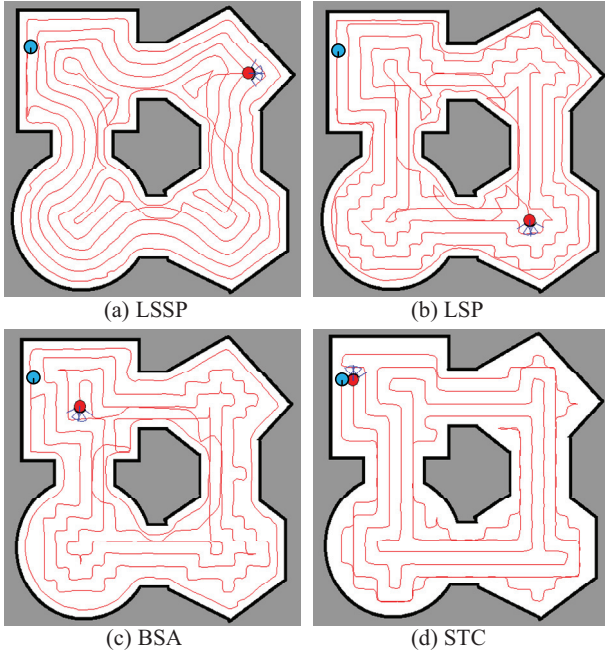


Figure 9. Coverage paths of the four algorithms, LSSP, LSP, BSA, and STC. The blue and red circles represent the initial and final pose of the robot, respectively.

BSA was lower than LSSP or LSP because BSA has no way of completely covering partially-covered cells [15], emerging from the obstacle-side-following behavior. STC resulted in the lowest coverage because it does not have strategy to cover partially-occupied cells. For the overlap rate, BSA recorded the lowest value. However, BSA sacrifices coverage of the partially-covered cells. On the other hand, LSP overlaps many regions at the cost of coverage. In spite of the highest coverage, LSSP achieved significantly lower overlap than LSP thanks to the high-resolution grid-map representation. For all the quantities, LSSP achieved the lowest standard deviations. This fact demonstrates the robustness of LSSP about the initial robot pose or the shape of the target environment.

VI. CONCLUSION

We presented a complete-coverage algorithm based on high-resolution grid-map representation and spiral motion, which generates smooth coverage paths. Thanks to the smoothness, the proposed LSSP algorithm achieves robustness against the initial robot pose or the shape of the target environment. It can not only increase coverage, but also reduce the overlap, which leads to time and energy saving. However, LSSP requires more memory for maintaining the grid-map than other compared algorithms. This may be problematic when embedded in low cost robots. Also, coverage at sharp-turning-points is a remaining problem. Our future work is to solve these problems and experiment with an embedded low-cost cleaning robot.

TABLE I. MEAN AND STD. (N=9) OF THREE QUANTITIES MEASURED DURING THE PROCESS OF FOUR ALGORITHMS IN THREE CHALLENGING ENVIRONMENTS.

Algorithm	Quantity					
	Turning angle per unit moving distance (deg/m)		Coverage rate (%)		Overlap rate (%)	
	mean	std.	mean	std.	mean	std.
LSSP	81	12	97.6	0.4	40.4	4.1
LSP	117	28	97.5	0.6	51.7	6.9
BSA	136	31	94.7	0.5	35.1	6.0
STC	122	38	78.3	4.2	40.2	9.3

REFERENCES

- [1] I. Ulrich, F. Mondada, and J.-D. Nicoud, "Autonomous vacuum cleaner," *Robotics and Autonomous Systems*, 19(3-4), pp.233-244, 1997.
- [2] Y. Huang, Z. Cao, and E. Hall, "Region filling operations for mobile robot using computer graphics," in *Proc. ICRA'86*, pp. 1607-1614.
- [3] J. D. Nicoud, and M. K. Habib, "The Pemex-B autonomous demining robot: perception and navigation strategies," in *Proc. IROS'95*, pp. 419-424.
- [4] M. Ollis, and A. Stentz, "First results in vision-based crop line tracking," in *Proc. ICRA'96*, pp. 951-956.
- [5] D. Gage, "Randomized search strategies with imperfect sensors," in *Proc. SPIE Mobile Robots VIII*, Boston, MA, 1993, pp. 270-279.
- [6] A. Zelinsky, R. A. Jarvis, J. C. Byrne, and S. Yuta, "Planning paths of complete coverage of an unstructured environment by a mobile robot," in *Proc. Int. Conf. Advanced Robotics*, 1993, pp. 533-538.
- [7] Y. Gabriely, and E. Rimon, "Spiral-STC: An on-line coverage algorithm of grid environments by a mobile robot," in *Proc. ICRA'02*, pp. 954-960.
- [8] H. Choset, "Coverage for robotics – A survey of recent results," *Annals of Mathematics and Artificial Intelligence*, 31(1-4), pp. 113-126, 2001.
- [9] S. V. Spires, and S. Y. Goldsmith, "Exhaustive geographic search with mobile robots along space-filling curves," *Collective Robotics, Lecture Notes in Computer Science*, 1456, pp. 1-12, 1998.
- [10] S. Hert, S. Tiwari, and V. Lumelsky, "A terrain-covering algorithm for an AUV," *Autonomous Robots*, 3, pp. 91-119, 1996.
- [11] H. Choset, E. Acar, A. Rizzi, and J. Luntz, "Exact cellular decompositions in terms of critical points of morse functions," in *Proc. ICRA'00*, pp. 2270-2277.
- [12] E. U. Acar, H. Choset, and J. Y. Lee, "Sensor-based coverage with extended range detectors," *IEEE Trans. Robotics*, 22(1), pp. 189-198, 2006.
- [13] E. Garcia, and P. Gonzalez, "Mobile-robot navigation with complete coverage of unstructured environments," *Robotics and Autonomous Systems*, 46(4), pp. 195-204, 2004.
- [14] E. Gonzalez, O. Alvarez, Y. Diaz, C. Parra, and C. Bustacara, "BSA: A complete coverage algorithm," in *Proc. ICRA'05*, pp. 2040-2044.
- [15] Y. H. Choi, T. K. Lee, S. H. Baek, and S. Y. Oh, "Online complete coverage path planning for mobile robots based on linked spiral paths using constrained inverse distance transform," in *Proc. IROS'09*, pp. 5788-5793.
- [16] J. S. Oh, Y. H. Choi, J. B. Park, and Y. F. Zheng, "Complete coverage navigation of cleaning robots using triangular-cell-based map," *IEEE Trans. Industrial Electronics*, 51(3), pp. 718-726, 2004.
- [17] D. Hearn, and M. P. Baker, *Computer Graphics*. New Jersey: Prentice-hall, 1997, pp. 315-350.